HOTEL RESERVATION SYSTEM

Hotel Reservation Project Handbook

Project	Hotel Reservation System
Document	Hotel Reservation Project Handbook
Document ID	PROJ-001
Version	1.0
Status	Active
Owner	Product & Engineering
Last Updated	2026-09-13

Official readable project artifact. Editable source is maintained under `docs/_source/`.

Contents

- Document control
- Executive summary
- Product vision and problem
  - Product vision
  - Problem statement
  - Product Goal
- Primary users
  - Guest / customer
  - Hotel administrator / staff
- Core user journeys
  - Room reservation journey
  - Staff reservation journey
  - Accommodation package journey
- V1 scope
- Explicitly out of scope for V1
- Confirmed business rules
- Technology stack
- Repository strategy
- High-level system architecture
- Modular Monolith principles
- Important quality attributes
  - Correctness and data integrity
  - Maintainability
  - Security and privacy
  - Reliability and operability
- Reservation integrity principles
- Environment strategy
  - Local development
  - Staging
  - Production
- Current project status
- Open and deferred decisions
- Project document map
- Documentation lifecycle

Document control

Version	Date	Change	Author
1.0	2026-09-13	Initial project handbook established before Sprint 1 implementation	Product & Engineering

Executive summary

The Hotel Reservation System is a greenfield hotel website and direct reservation management product. It will provide a professional public experience for prospective guests and a secure operational interface for hotel staff. Guests will be able to discover the hotel, browse room types, search real availability, make a direct reservation without creating an account, receive a booking reference and confirmation, and pay at the hotel on arrival.

The first delivery priority is the **room-booking integrity path** rather than marketing breadth: room catalogue, availability, conflict-safe reservation creation, and booking confirmation. Packages, broader marketing content, and richer administration remain part of V1 but follow the core booking slice.

The solution uses a **Next.js frontend**, a **Node.js/Express REST API**, and **PostgreSQL**, deployed on **Railway**. The backend is a **feature-first Modular Monolith**: one application and one deployment, internally organized by business capability with explicit module boundaries.

Product vision and problem

Product vision

Provide the hotel with a dependable direct-booking channel that is easy for guests to use and straightforward for hotel staff to operate, without introducing unnecessary payment, account, channel-manager, or multi-property complexity in V1.

Problem statement

The hotel needs a professional digital presence and direct reservation capability. Guests need a low-friction path to understand the property, identify a suitable available room, reserve it, and receive confirmation. Hotel staff need a secure system to manage inventory, reservations, pricing, packages, and booking/payment statuses while preserving reservation and guest-data integrity.

Product Goal

Deliver a production-ready hotel website and reservation system that enables guests to discover the hotel, search real room availability, make a direct Pay-at-Hotel reservation, and receive confirmation, while authorized hotel staff can securely manage rooms, inventory, reservations, pricing, and packages.

Primary users

Guest / customer

A guest is an unauthenticated public user. V1 does not require a customer account. The guest can browse the hotel and room catalogue, search by stay dates and occupancy, reserve one room, receive a reference and confirmation, browse accommodation packages, and submit package reservations when that capability is delivered.

Hotel administrator / staff

Hotel staff authenticate to a protected admin experience. V1 uses one administrator/staff permission level. Staff manage room types, physical rooms, prices, availability, blocks, reservations, reservation/payment statuses, and packages.

Core user journeys

Room reservation journey

1. Guest selects check-in date, check-out date, and guest count.
2. System validates the request and returns room types with real available physical inventory.
3. Guest reviews a room type and booking details.
4. Guest enters contact details and submits the reservation.
5. Backend revalidates availability and capacity.
6. Backend atomically allocates an eligible physical room.
7. Backend calculates nights and the accepted EGP price server-side.
8. Pending reservation, booking reference, allocation, and price snapshot are persisted.
9. Guest sees a confirmation page without internal room-number disclosure.
10. Confirmation email is sent when transactional email is enabled.

Staff reservation journey

1. Staff signs in securely.
2. Staff views/searches reservations and guest details.
3. Staff updates reservation and payment status as operational work progresses.

4. Cancelling or otherwise releasing a reservation makes inventory available again according to the business rules.

Accommodation package journey

A package is an accommodation offer that includes a stay. It is associated with one or more eligible room types and consumes the same physical room inventory as a standard room reservation. Package booking will reuse the same inventory/reservation integrity rules rather than implement a second availability system.

V1 scope

V1 includes:

- Responsive public hotel website in English.
- Home, About, Rooms, Room Details, Facilities/Services, Packages/Offers, Package Details, Gallery, and Contact/Location experiences.
- SEO-friendly public pages.
- Room-type catalogue with photos, description, amenities, capacity, and EGP nightly price.
- Date/occupancy availability search validated by the backend.
- Guest reservation flow without an account.
- Conflict-safe physical room allocation behind the room-type booking experience.
- Booking reference, confirmation page, and transactional confirmation email.
- Pay at Hotel / Cash on Arrival; no online payment collection.
- Protected staff/admin access.
- Room type, physical room, pricing, capacity, availability/block management.
- Reservation list/detail, booking status, and payment status management.
- Accommodation packages/offers and package reservations.
- Persistent media storage for hotel/room/package/gallery assets.
- Railway staging and production deployment foundation.

Explicitly out of scope for V1

- Online card or wallet payments.
- Guest accounts, self-service booking modification, or self-service cancellation.
- Loyalty/rewards.
- Native mobile apps.
- Booking.com, Expedia, Airbnb, or other OTA synchronization.
- External channel manager integration.
- Advanced analytics/reporting.
- Multi-property management.
- Multi-currency support.

- Arabic/multilingual support.
- Seasonal, weekend, holiday, or promotional date-range rate plans.
- Multi-room reservations in a single booking.
- Standalone service-only packages.
- Advanced administrator role-based access control.

Confirmed business rules

Area	Confirmed V1 rule
Guest booking unit	Guest selects a **Room Type**, not a physical room.
Physical allocation	Backend assigns an eligible physical room internally. Public booking responses do not expose the room number.
Reservation cardinality	One reservation = one room.
Pricing	One current base nightly price per room type. Total = server-calculated nightly price x number of nights.
Price history	Accepted nightly price, nights, currency, and total are stored as a reservation price snapshot.
Currency	EGP only.
Stay interval	Reservation dates use **[check-in, check-out)** so checkout-day turnover is allowed.
Initial reservation status	Pending.
Inventory consumption	A successfully created Pending reservation immediately consumes inventory.
Release of inventory	Inventory is released when the reservation is cancelled/rejected or otherwise explicitly released by staff.
Payment	Pay at Hotel / Cash on Arrival. Payment state is Unpaid/Pay at Hotel or Paid.
Guest identity	No guest account in V1; reservation stores a guest/contact snapshot.
Staff permissions	One administrator/staff permission level in V1.
Packages	Accommodation packages include a stay, have a fixed package price/duration, are tied to eligible room types, and consume the same physical inventory.
Booking modification	Guests contact the hotel; staff perform changes in the admin dashboard.

Area	Confirmed V1 rule
Language	English only for initial V1.

Technology stack

Concern	Decision
Frontend	Next.js + TypeScript
Backend	Node.js + Express + TypeScript
API style	REST, versioned business namespace `/api/v1`
Database	PostgreSQL
Repository	npm workspaces monorepo
Applications	`apps/web`, `apps/api`
Backend architectural style	Feature-first Modular Monolith
Hosting	Railway
Payment	No gateway in V1
Media	Persistent object/media storage; provider deferred
Transactional email	Provider deferred

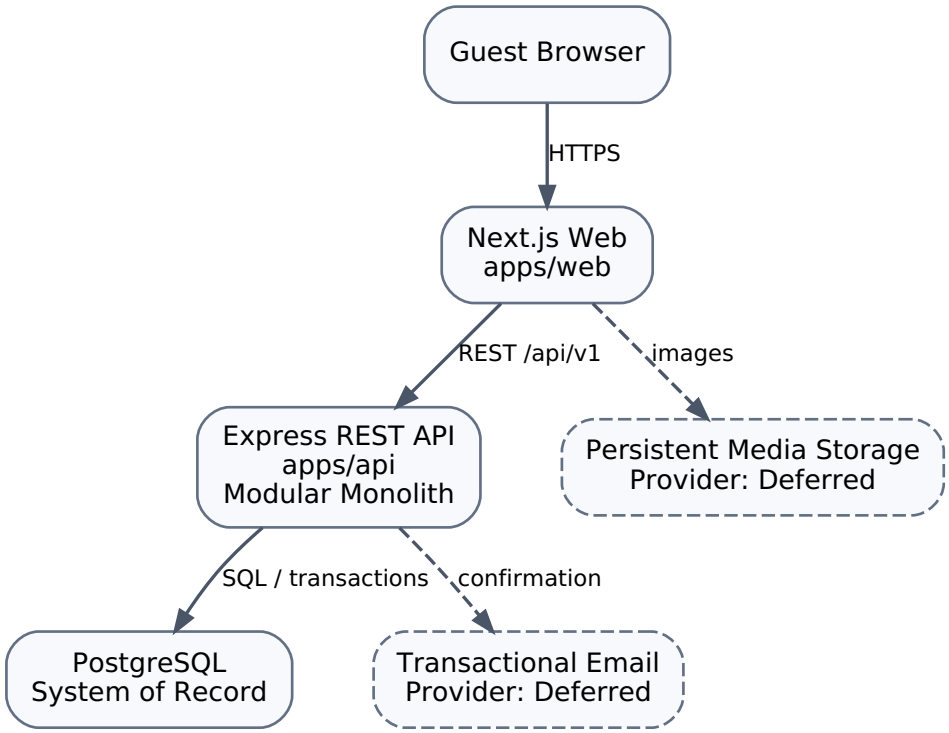
Repository strategy

The repository is a single npm-workspaces monorepo. Runtime dependencies remain owned by the application that uses them instead of accumulating at the repository root.

```
hotel-reservation/
├── apps/
│   ├── web/         # Next.js
│   └── api/         # Express modular monolith
├── docs/            # official PDFs + reproducible editable sources
├── .github/         # CI workflows when introduced
├── package.json
└── workspace configuration
```

Shared monorepo packages are introduced only when actual cross-application reuse justifies them. Empty `packages/ui`, `packages/types`, or similar placeholders are intentionally avoided.

High-level system architecture



System topology

The browser interacts with the Next.js application. The web application uses the Express REST API for hotel-domain behavior. PostgreSQL is the final system of record for inventory and reservation state. Third-party email and persistent media providers are integration points but are not yet selected.

Modular Monolith principles

The backend remains **one Express application, one deployable backend service, and one PostgreSQL database**. Business code is organized by capability, such as `rooms`, `reservations`, `auth`, `packages`, `media`, and `notifications` when those capabilities become necessary.

Modules own their capability-specific code. A module must not freely import another module's internal implementation. Cross-module behavior must be exposed intentionally. Shared technical code is permitted only when it is genuinely cross-cutting; reservation-specific or room-specific business logic stays in its owning module.

The architecture explicitly avoids premature enterprise abstractions such as microservices, event buses, CQRS, generic base repositories, dependency-injection frameworks, and one package per module unless a future requirement clearly justifies them.

Important quality attributes

Correctness and data integrity

Reservation availability and allocation correctness are higher priority than visual breadth. PostgreSQL is the final source of truth for overlapping inventory use. Reservation creation must remain correct under concurrent requests.

Maintainability

Feature-first modules, explicit module boundaries, small vertical increments, repeatable migrations, focused automated tests, and living documentation should allow the system to grow without requiring a structural rewrite.

Security and privacy

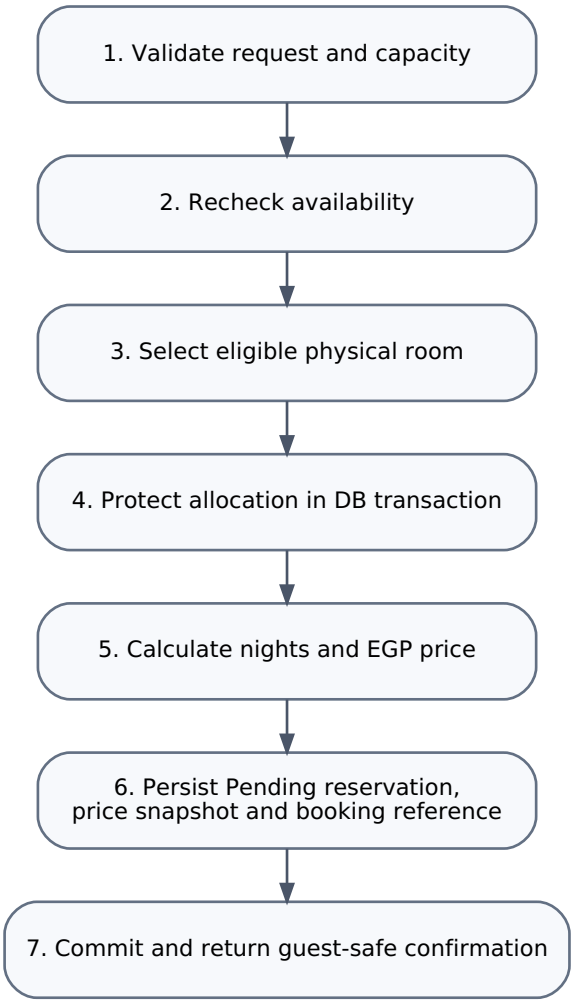
- Admin operations require authentication and authorization.
- Passwords are hashed using an appropriate password-hashing algorithm.
- Server secrets are never exposed to the frontend or committed to source control.
- Input is validated on the backend; frontend validation is only a usability aid.
- Guest PII is minimized and must not be unnecessarily logged.
- Public booking flows never trust client-submitted room IDs, prices, totals, or stale availability results.
- Production traffic uses HTTPS through the hosting platform/domain configuration.

Reliability and operability

CI, repeatable deployments, structured server logs, health checks, backup/recovery awareness, error handling, and basic monitoring are part of delivery rather than postponed to a separate phase.

Reservation integrity principles

The booking engine uses the interval **[check-in, check-out)**. This permits a room to be checked out and checked in again on the same calendar date.



Reservation transaction flow

Reservation creation must conceptually perform the following inside a safe persistence strategy:

1. Validate the request server-side.
2. Validate room-type guest capacity.
3. Recheck availability.
4. Find an eligible physical room for the full stay.
5. Protect the allocation inside a database transaction.
6. Prevent overlapping active reservations at the database level.
7. Calculate nights and price server-side.
8. Store the accepted price snapshot.
9. Create the Pending reservation and unique booking reference.
10. Commit.
11. Return only guest-safe confirmation data.

Two competing requests for the only remaining physical room over overlapping dates must not both succeed.

Environment strategy

Local development

Developers run the Next.js and Express applications from the npm workspace and connect to a development PostgreSQL instance once database work begins. Local secrets are stored in ignored environment files; tracked example configuration contains names and harmless examples only.

Staging

Sprint increments are deployed to an inspectable Railway staging environment. Staging is the integration target for Sprint Review behavior and production-like configuration checks.

Production

Production will run on Railway and connect to the hotel domain when launch readiness is complete. Production secrets are held in Railway/environment configuration, not in the repository. Launch readiness includes database backup/recovery checks, domain/HTTPS verification, email configuration, smoke testing, and operational logging/monitoring.

Current project status

Current phase: Sprint 1 ready to start; implementation has not begun.

Sprint 1 selected PBIs: PBI-001 Room catalogue/details, PBI-002 Availability search, PBI-003 Conflict-safe reservation creation, PBI-004 Booking confirmation.

Capacity candidate: PBI-005 secure staff sign-in and reservation inspection may be pulled only after the Sprint Goal is safe.

Current development task: Sprint 1 Task 1 - establish the npm-workspaces repository/application skeleton, Next.js web app, Express modular-monolith shell, `/health` endpoint, environment configuration, and frontend-to-API health integration. PostgreSQL and business modules are explicitly excluded from Task 1.

Open and deferred decisions

Decision	State	Notes
Transactional email provider	DEFERRED	Select when PBI-006 approaches implementation.
Persistent media provider	DEFERRED	Select before durable room/package/gallery uploads.

Decision	State	Notes
Map/contact implementation	DEFERRED	Does not affect core booking integrity.
ORM / SQL access approach	DEFERRED	Decide immediately before PostgreSQL persistence work; must support required integrity constraints and transactions.
Node.js runtime version	OPEN FOR TASK 1	Pin/document a supported version when the repository is created.
Detailed admin UX	PROVISIONAL	Refine when PBI-005/management PBIs approach implementation.
Advanced observability provider	DEFERRED	Basic structured logging and platform monitoring are sufficient initially.
Seasonal/rate-plan architecture	DEFERRED	Current base-price design must not make future extension impossible.

Project document map

Document	ID	Purpose
Hotel Reservation Project Handbook	PROJ-001	Primary onboarding and project source-of-truth summary.
Product Backlog	PROD-001	Single ordered Product Backlog and item readiness/status.
System Architecture	ARCH-001	Current system boundaries, technical direction, integrity/ security strategy.
ADR-001 Modular Monolith	ADR-001	Record of the backend architectural-style decision.
Sprint 01 Plan	SPRINT-001	Sprint Goal, selected PBIs, implementation tasks, risks, DoD, testing and review plan.
Claude Code Engineering Guide	ENG-001	Guardrails for AI-assisted development while the developer remains accountable.

Documentation lifecycle

PDFs under `docs/` are the official readable artifacts. Editable Markdown and diagram sources are maintained under `docs/_source/`, with `docs/build_docs.py` used to regenerate the PDFs.

Material changes create a new document version; Sprint plans are retained as historical records after the Sprint closes.